

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 10

NOMBRE COMPLETO: José Eduardo Martínez García

Nº de Cuenta: 320304679

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 04/11/2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas

Para esta práctica trabajé sobre la base del ejercicio de practica numero 10, por ello conservé el avión y su humo, y agregué las animaciones del holocrón y de la nave espacial. Primero preparé los modelos en Blender de los modelos que ya había descargado en entregables anteriores, por ejemplo, el cohete en un previo y el holocron que era el que el profesor había proporcionado, además busqué una lauchpad (plataforma de lanzamiento) para complementar al cohete que había descargado y después los cargué en OpenGL.

1.- Holocrón hace la animación del GIF

Aquí quise reproducir la secuencia de apertura, giro y cierre del holocrón que mostró el profesor en el GIF. Traté de acercarme al movimiento usando operaciones básicas (incrementos de ángulo y de distancia) y un control de fases o partes de las que se compone la animación requerida. Lo que se ve es esto: una esquina gira 45 grados alrededor de la dirección de su punta, en 0.6 segundos. Cuando termina, las otras siete esquinas giran sus 45 grados al mismo tiempo, también en 0.6 segundos. Después las ocho siguen girando y se alejan cuatro unidades del centro durante cuatro segundos, y al final regresan otros cuatro segundos sin invertir el sentido del giro. Cada pieza completa 360 grados, vuelve a encajar y la secuencia se repite sola. Cada esquina se mueve sobre su propia diagonal, es decir, las de abajo se alejan hacia abajo y las de arriba hacia arriba, así que no sube todo sobre el eje Y y tampoco hay una trayectoria en espiral.

En Blender, el modelo tenía el cuerpo central y las ocho esquinas separadas, pero no tenía un acabado de texturas listo. Busqué opciones y referencias además del gift que ya tenia, propuse el azul con detalles dorados y trabajé el mapeo UV. Conservé los cuadrados, círculos y triángulos del relieve, incluyendo las caras que quedan debajo de las pirámides triagnulares cuando se levantan estas. El resultado es opaco. También revisé los orígenes y las coordenadas de cada objeto para poder volver a armar el holocrón, y exporté cada pieza con su OBJ, MTL y texturas. Las direcciones de giro las retomé de la práctica 5.

```
const glm::vec3 PIVOTE_ESQUINA[8] = { ... };
const glm::vec3 EJE_ESQUINA[8] = {
    glm::vec3( 1.0f,  1.0f,  1.0f),
    glm::vec3(-1.0f,  1.0f,  1.0f),
    glm::vec3( 1.0f, -1.0f,  1.0f),
    ...
};
```

PIVOTE_ESQUINA coloca cada esquina en su lugar y EJE_ESQUINA es el vector de la diagonal, o sea, la dirección de la punta, que se usa como eje de giro. Con normalize se conserva solo la dirección, así que todas las esquinas pueden usar la misma distancia de separación.

```
tiempoHolocron += dt;
if (faseHolocron == 0)
    anguloEsquina[0] += 45.0f * dt / 0.6f;
else if (faseHolocron == 1)
    for (int i = 1; i < 8; i++)
        anguloEsquina[i] += 45.0f * dt / 0.6f;
```

En actualizaHolocron el tiempo se va acumulando. En la primera fase solo se mueve la esquina 0 y, cuando llega a sus 45 grados, el contador de fase cambia y el ciclo mueve las otras siete a la vez. No giran una tras otra: giran las siete juntas, con el mismo incremento.

```
anguloEsquina[i] += 157.5f * dt / 4.0f;      // apertura y regreso
distanciaEsquina[i] += signo * 4.0f * dt / 4.0f;  // +1 al abrir, -1 al regresar
```

$$\text{incremento angular inicial} = 45 \cdot \frac{dt}{0.6}$$

$$\text{incremento angular de apertura o regreso} = 157.5 \cdot \frac{dt}{4}$$

$$\text{incremento de separación} = \pm 4 \cdot \frac{dt}{4}$$

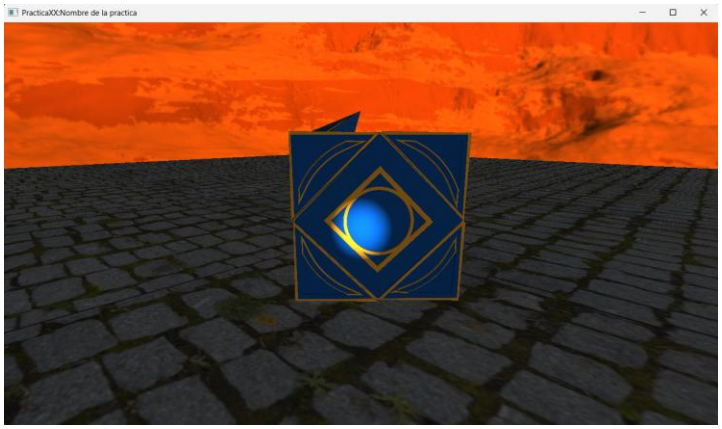
En las fases de apertura y de regreso las esquinas giran 157.5 grados cada una, así que con los 45 iniciales completan los 360 (45 + 157.5 + 157.5). La separación cambia de signo al regresar, pero el giro mantiene el mismo sentido.

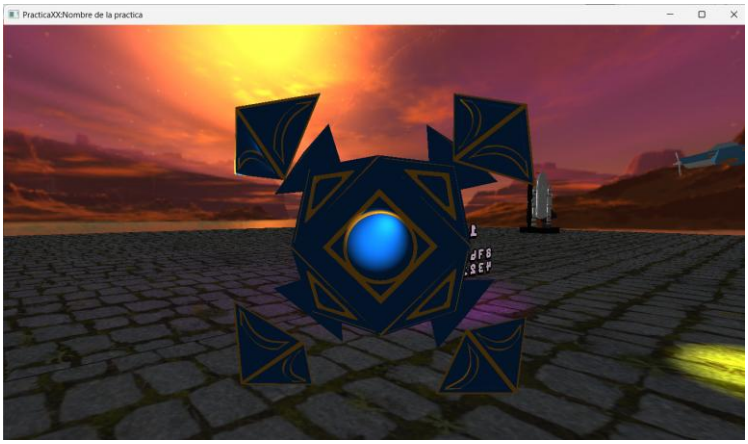
```
modelHolocron = modelHolocronAux;
modelHolocron = glm::translate(modelHolocron, PIVOTE_ESQUINA[i] +
    glm::normalize(EJE_ESQUINA[i]) * distanciaEsquina[i]);
modelHolocron = glm::rotate(modelHolocron, anguloEsquina[i] * toRadians, EJE_ESQUINA[i]);
```

Aquí está la jerarquía de matrices. Copio modelHolocronAux (la matriz del cuerpo central), traslado cada esquina a su pivote más la distancia sobre su diagonal y después aplico rotate sobre el eje de esa misma diagonal. La posición depende de la traslación y la rotación se aplica a la pieza sobre su eje.

```
actualizaHolocron(segundosAvion);
...
dibujaHolocron();
```

Las dos llamadas están en el ciclo principal. La animación es automática, así que no necesita ninguna tecla: actualizaHolocron mueve los valores en cada cuadro y dibujaHolocron dibuja el cuerpo central y las ocho esquinas.





2.- La nave espacial despega y sigue una trayectoria a partir de ecuaciones y algoritmos

Primero cuento cómo se mueve. La nave y sus tres motores están colocados sobre una plataforma fija, y el conjunto se orientó 90 grados para que el movimiento horizontal quedara hacia el interior del piso. Con la tecla 4 inicia la secuencia y, si la vuelvo a presionar, se reinicia. El conjunto sube verticalmente 120 unidades de la escena sobre su posición inicial durante 12 segundos. A las tres cuartas partes del ascenso (90 unidades, a los ocho segundos) se separan los dos motores laterales, que caen en vertical mientras la nave sigue subiendo. Al completar el ascenso se separa el motor central y también cae. La nave sola avanza hacia el interior con una caída parabólica; en el código el avance horizontal es sobre +X y Z se queda constante, sin ningún giro progresivo sobre el eje vertical. Antes de llegar al piso pasa a una recuperación curva de 100 unidades horizontales en 3.5 segundos, con la punta al frente y las alas niveladas. Después del contacto sigue un rodaje de 27 unidades durante tres segundos, bajando la velocidad hasta detenerse. Las distancias son unidades de la escena y no las tomé como metros; el 120 es un incremento sobre la posición inicial, no la coordenada Y del modelo.

La preparación de los modelos fue así: la plataforma se quedó como ambiente y la nave, el motor central y los dos laterales se exportaron por separado. En Blender separé las partes con Split, y revisé los cortes, los orígenes y las coordenadas para que las piezas se pudieran armar sin desplazamientos extraños. Las UV y las texturas las conservé o ajusté para que el conjunto se viera como el original al importarlo.

```
cargaHolocronCohete();
...
glm::mat4 matrizMotorCohete(int motor);
...
modelNaveCohete = glm::translate(glm::mat4(1.0f), POS_COHETE);
modelNaveCohete = glm::rotate(modelNaveCohete, 90.0f * toRadians, glm::vec3(0,1,0));
```

cargaHolocronCohete hace los LoadModel de cada pieza, construyeCohete arma el conjunto y matrizMotorCohete da la matriz de cada motor. Todos arrancan en su posición inicial con el mismo giro fijo de 90 grados. La plataforma tiene su propia transformación, y POS_COHETE permite ajustar el conjunto de la nave sin mover la plataforma.

```
float alturaCohete(float t)
{
   float u = glm::clamp(t / 12.0f, 0.0f, 1.0f);
   return 120.0f * (1.0f - cos(PI_AVION * u)) / 2.0f;
}
```

$$u = clamp(t/12, 0, 1)$$

$$altura(t) = 120 \cdot \frac{[1 - cos(\pi u)]}{2}$$

El coseno hace que el despegue arranque y termine suave en lugar de golpe, y clamp mantiene u entre 0 y 1. Esta función se usa solo para el desplazamiento vertical del despegue.

```
float caida = alturaSeparacion - 0.5f * 9.8f * tSep * tSep;
caida = glm::max(caida, 0.0f);   // el motor no atraviesa el piso
```

$$caída(t) = altura\ de\ separación - 0.5 \cdot 9.8 \cdot tiempo_desde_separación^2$$

Cada motor guarda la altura a la que se separó y de ahí cae con gravedad. X y Z se quedan fijas después de separarse, y max evita que el motor pase del piso. Los laterales se separan a los ocho segundos y el central a los doce.

```
float avance = 32.0f * t;
float altura = alturaInicioCaida - 0.5f * 9.8f * t * t;
float duracionParabola = sqrt(2.0f * (alturaInicioCaida - alturaRecuperacion) / 9.8f);
```

$$avance\ horizontal = 32 \cdot t$$

$$altura = altura\ al\ iniciar\ la\ caída - 0.5 \cdot 9.8 \cdot t^2$$

Si combino un avance horizontal uniforme con un descenso que crece con t², sale una parábola. duracionParabola usa sqrt para calcular cuándo debe empezar la recuperación, más o menos 55 unidades por encima del nivel final de aterrizaje.

```
glm::vec3 recuperacionCohete(float u)
{
   float u2 = u * u, u3 = u2 * u;
   return (2*u3 - 3*u2 + 1) * P0 + (u3 - 2*u2 + u) * T0 +
          (-2*u3 + 3*u2) * P1 + (u3 - u2) * T1;
}
```

$$H(u) = (2u^3 - 3u^2 + 1)P0 + (u^3 - 2u^2 + u)T0 + (-2u^3 + 3u^2)P1 + (u^3 - u^2)T1$$

Es una curva cúbica de Hermite. P0 y P1 son la posición inicial y final de la recuperación, y T0 y T1 controlan cómo entra y sale la nave de la curva. La tangente vertical final es cero, para que llegue al piso de forma horizontal.

```
typedef struct { float angulo; float inclinacionInc; } FRAME_COHETE;
// angulos: 0, 95, 125 y 90
```

```
inclinacionInc = (anguloSiguiente - anguloActual) / 90.0f;
```

$$incremento = \frac{ángulo\ siguiente - ángulo\ actual}{90 pasos}$$

$$suavizado\ del\ último\ intervalo: 3u^2 - 2u^3$$

Aquí retomé la idea de la estructura de keyframes de clase, pero solo para la inclinación de la nave, con cuatro ángulos: 0, 95, 125 y 90. En el último intervalo uso $3u^2 - 2u^3$ para que la inclinación sea suave al aterrizar. La posición no está guardada en keyframes: sale de las ecuaciones anteriores.

```
modelNaveCohete = glm::rotate(modelNaveCohete, 90.0f * toRadians, glm::vec3(0,1,0));
modelNaveCohete = glm::rotate(modelNaveCohete, inclinacion * toRadians, glm::vec3(0,0,1));
```

Son dos rotaciones. La de 90 grados fija el rumbo desde el principio y la otra cambia únicamente la inclinación. Así la nave no aterriza de lado y conserva las alas niveladas.

```
float rodaje = 27.0f * (2.0f * u - u * u);
```

Con $27 \cdot (2u - u^2)$ la nave avanza y va frenando en el piso hasta detenerse. Basta con eso para el rodaje.

```
// Emisor de cada motor y edad de las particulas
actualizaHumoCohete(segundosAvion);
...
for (auto& particula : humoCohete)
    if (particula.edad < VIDA_HUMO) visibles.push_back(&particula);
```

Para el humo reutilicé la base que ya funcionaba con el avión, así que no tuve que crear otro sistema. En actualizaHumoCohete cada motor es un emisor, y las matrices hacen que el humo salga de ellos y los siga aunque ya se hayan desprendido. dibujaHumo actualiza la edad y escoge las partículas visibles. Uso la misma imagen humo.png, el mismo shader, la mezcla alfa, el billboard y el orden de dibujo. Cada emisión genera una nube por motor; el intervalo propio de la nave es 0.4 segundos y su tamaño es 6.5. Cada nube dura dos segundos y luego deja de dibujarse. Estos valores son solo de la nave: el humo del avión no cambió.

```
// Window.h
bool getSolicitudCohete() { return solicitudCohete; }
bool solicitudCohete;

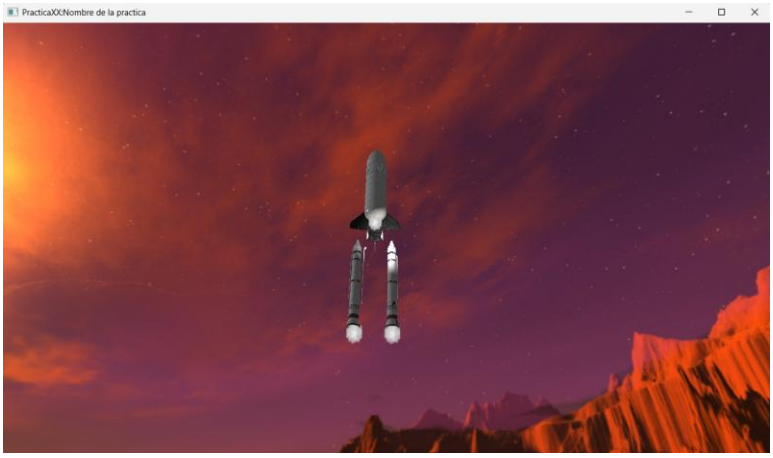
// Window.cpp (dentro de GLFW_PRESS)
if (key == GLFW_KEY_4)
    theWindow->solicitudCohete = true;

// main
```

```
actualizaCohete(segundosAvion);
...
dibujaCohete();
```

En Window.h agregué el getter y la variable de solicitud, que se inicializa en los dos constructores. En Window.cpp el bloque de GLFW_KEY_4 va dentro de GLFW_PRESS. Al final, el main solo llama a actualizaCohete y a dibujaCohete.









2. Problemas presentados

- El holocrón no tenía las texturas listas y me costó encontrar una apariencia parecida a la referencia. Busqué opciones y propuse el azul con bordes dorados en Blender. También revisé el mapeo UV y las caras que quedaban debajo de las pirámides, porque ahí aparecían partes del mapa que no correspondían al relieve.
- Se me complicó colocar las esquinas y encontrar sus ejes para reproducir el movimiento del GIF. Retomé las direcciones de la práctica 5, ajusté los pivotes y separé la animación por fases. Así pude hacer que girara primero una esquina, después las otras siete juntas y al final todas se alejaran y regresaran sin perder su posición.
- La nave venía como un modelo completo y separar sus partes en Blender me tomó varios intentos. Usé Split para separar los componentes que necesitaba y revisé los cortes y los orígenes antes de expotarlos. Al cargar la nave y el holocrón en OpenGL, ajusté las coordenadas y las matrices para que los objetos separados volvieran a cuadrar como un solo conjunto.
- Me costó elegir una trayectoria que pudiera implementar con funciones básicas y que terminara en un aterrizaje natural. Lo resolví probando coseno, caída libre, ecuaciones parabólicas, interpolación por incrementos y la curva cúbica de Hermite. Después ajusté los keyframes de inclinación y los ejes de giro para que la nave llegara con las alas niveladas y no de lado.

3. Conclusión

- Los ejercicios se me hicieron de dificultad media. Lo que más me costó fue coordinar las piezas y combinar las etapas del movimiento sin cambios bruscos.
- Comprendí mejor cómo usar traslaciones, rotaciones, funciones matemáticas y pocos keyframes para construir una animación sin guardar una gran cantidad de posiciones.
- Reutilizar el humo del avión me permitió concentrarme en los emisores de la nave y en hacer que siguieran a cada motor durante la separación.